智能测试管理平台 · 完整系统设计

版本：2.1
日期：2026-02-24
作者：资深Go开发专家

---

1. 系统概述

智能测试管理平台旨在为企业级研发团队提供一站式的测试全生命周期管理，深度融合AI能力，将产品需求文档（PRD）、Figma设计稿等非结构化数据转化为可执行的测试用例，实现需求即测试、测试即代码。平台采用Go微服务架构，通过RAG（检索增强生成）技术构建领域知识库，利用大语言模型辅助测试设计，大幅提升测试用例的编写效率与质量。

1.1 核心能力

- **项目管理**：多项目隔离，支持模块化组织测试资产。项目配置独立为`project_config`表，支持键值对扩展。
- **知识库构建**：支持PRD（Word/PDF/Markdown）、Figma设计稿、OpenAPI规范等自动解析、分块、向量化。
- **AI用例生成**：基于用户指令，通过RAG检索相关需求片段，调用大模型生成结构化用例草稿，支持人工审核确认。
- **用例管理**：用例的增删改查、版本历史、需求追溯、导入导出。
- **测试计划与执行**：计划编排、任务分配、结果记录、执行历史追溯。
- **质量分析**：用例覆盖率、需求覆盖矩阵、执行趋势、AI效能看板。

1.2 用户角色

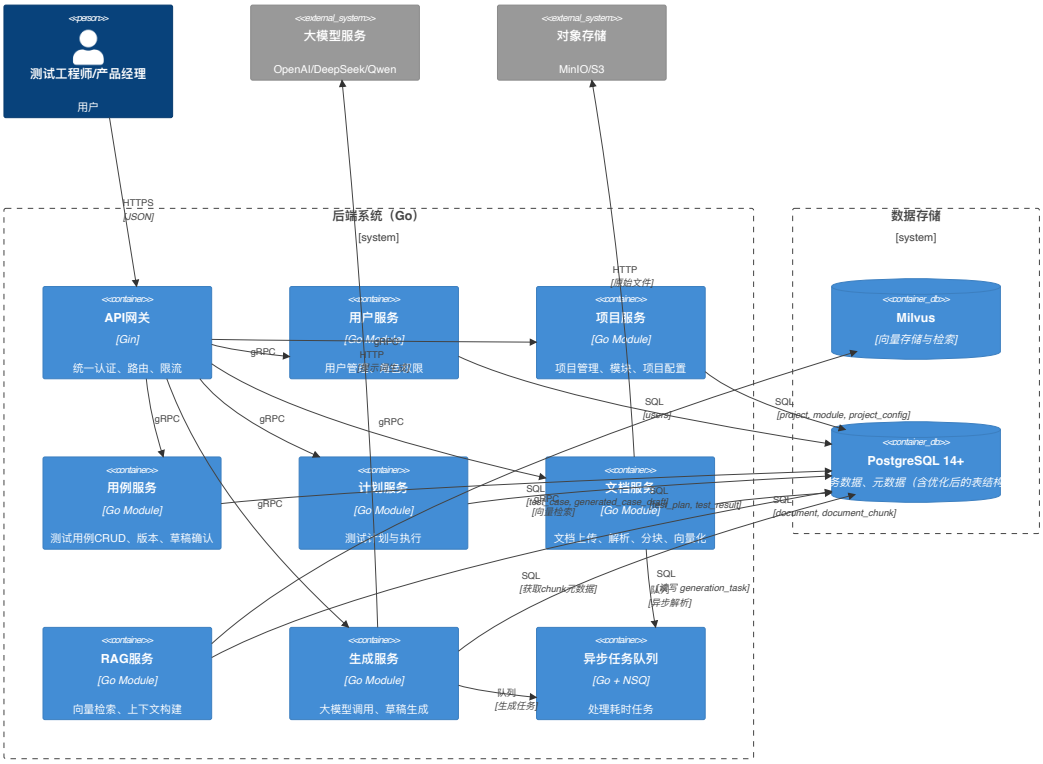
角色	权限描述
超级管理员	系统配置、租户管理、模型提供商配置、项目创建、成员管理、知识库维护、生成任务发起
测试工程师	用例设计、草稿确认、测试计划执行、结果录入

---

2. 总体架构

2.1 系统架构图

系统整体架构图（集成RAG+大模型）



2.2 技术栈选型

领域	技术栈	说明
后端语言	Go 1.22+	高性能，适合微服务架构
Web框架	Gin	轻量级，中间件丰富
服务通信	gRPC + Protobuf	内部服务间高性能通信
数据库	PostgreSQL 14+	关系数据存储，支持JSONB与GIN索引
向量数据库	Milvus	专为向量检索优化，支持十亿级规模
对象存储	MinIO / AWS S3	存储原始文档、执行附件
大模型	DeepSeek	提供 embedding 与 text-generation
ORM	GORM	数据库操作（推荐GORM便于快速开发）
消息队列	RabbitMQ	异步处理文档解析、大模型调用
部署	Docker + Kubernetes	容器化部署，支持水平伸缩
监控	Prometheus + Grafana	服务可观测性
日志	EFK (Elasticsearch + Fluentd + Kibana)	集中日志管理

2.3 模块划分

系统采用领域驱动的微服务（或模块化单体）设计，各服务可独立部署：

- **api-gateway**：统一入口，认证、路由、限流。
- **user-service**：用户、角色、权限。
- **project-service**：项目、模块管理，**项目配置**（CRUD键值对）。
- **testcase-service**：测试用例、草稿、需求追溯。
- **testplan-service**：测试计划、执行结果。

- **document-service**：文档上传、解析、分块、向量化（调用embedding）。
- **rag-service**：向量检索、上下文组装。
- **generation-service**：调用大模型、生成任务管理。
- **worker**：异步任务处理器（可合并到各服务或独立）。

---

3. 数据模型详解

基于新SQL文件，以下为详细数据模型。

3.1 枚举类型

类型名	值域	用途
`case_status_enum`	`unexecuted`, `pass`, `block`, `fail`	测试用例状态
`case_type_enum`	`functionality`, `performance`, `api`, `ui`, `security`	用例类型
`plan_status_enum`	`draft`, `active`, `completed`, `archived`	测试计划状态
`priority_enum`	`P0`, `P1`, `P2`, `P3`	优先级
`result_status_enum`	`pass`, `fail`, `block`, `skip`	执行结果
`user_role_enum`	`super_admin`, `admin`, `normal`	用户角色

3.2 核心业务表

`project` 项目表

字段	类型	说明
id	uuid (PK)	
name	varchar(255) UNIQUE	项目名称
description	text	描述
created_at	timestamptz	
updated_at	timestamptz	

索引：无额外索引，主键和唯一键已满足。

`project_config` 项目配置表（新增）

字段	类型	说明
id	bigserial (PK)	自增主键，节省空间
project_id	uuid (FK -> project)	所属项目，ON DELETE CASCADE
key	varchar(255)	配置键
value	jsonb	配置值，支持结构化数据
description	text	配置项说明
created_at	timestamptz	
updated_at	timestamptz	
约束	UNIQUE(project_id, key)	项目内配置键唯一

索引：

- `idx_project_config_project_id`：加速按项目查询。
- `idx_project_config_key`：加速按键查询。

`module` 模块表

字段	类型	说明
id	uuid (PK)	
project_id	uuid (FK -> project)	所属项目，ON DELETE CASCADE
name	varchar(255)	模块名称

name	varchar(255)	模块名称
description	text	描述
约束	UNIQUE(project_id, name)	项目内模块名唯一

索引：`idx_module_project_id`。

`users` 用户表

字段	类型	说明
id	uuid (PK)	
username	varchar(32)	登录名
email	varchar(255) UNIQUE	邮箱
password	varchar(255)	加密后的密码
role	user_role_enum	角色
created_at	timestamptz	
updated_at	timestamptz	

索引：`idx_users_username`，`idx_users_email`。

`test_case` 测试用例表

字段	类型	说明
id	uuid (PK)	
module_id	uuid (FK -> module)	所属模块，ON DELETE CASCADE

user_id	uuid (FK -> users)	创建者，ON DELETE NO ACTION
number	varchar(32) UNIQUE	用例编号（业务唯一，如 `PROJ-MOD-20250224-001`）
title	varchar(255)	标题
preconditions	jsonb	前置条件数组 `["用户已登录", "商品库存>0"]`
steps	jsonb	步骤数组 `["步骤1", "步骤2"]`
expected	jsonb	预期结果对象 `{"result": "成功", "data": {...}}` 或字符串
ai_metadata	jsonb	AI相关元数据（引用的文档块ID列表、生成任务ID、置信度）
case_type	case_type_enum	用例类型
priority	priority_enum	优先级
status	case_status_enum	当前状态
created_at	timestamptz	
updated_at	timestamptz	

索引：

- `idx_test_case_module_id`，`idx_test_case_user_id`，`idx_test_case_number`。
- GIN索引：`idx_test_case_ai`（`ai_metadata`），`idx_test_case_steps`（`steps`）。

`test_plan` 测试计划表

字段	类型	说明
id	uuid (PK)	
project_id	uuid (FK -> project)	所属项目，ON DELETE CASCADE
user_id	uuid (FK -> users)	创建者，ON DELETE NO ACTION
name	varchar(255)	计划名称
status	plan_status_enum	状态
extra_config	jsonb	扩展配置（如环境信息）
created_at	timestamptz	

updated_at	timestamptz	

索引：`idx_test_plan_project_id`，`idx_test_plan_user_id`。

`test_result` 测试结果表

字段	类型	说明
id	uuid (PK)	
case_id	uuid (FK -> test_case)	执行的用例，ON DELETE CASCADE
plan_id	uuid (FK -> test_plan)	所属计划，ON DELETE CASCADE
executor_id	uuid (FK -> users)	执行者，ON DELETE NO ACTION
execute	result_status_enum	执行状态
result_details	jsonb	详细结果（日志URL、截图、断言）
executed_at	timestamptz	执行时间戳

索引：`idx_test_result_case_id`，`idx_test_result_plan_id`，`idx_test_result_executor_id`，`idx_test_result_executed_at`。

3.3 知识库与AI相关表

`document` 文档表

字段	类型	说明
id	uuid (PK)	
project_id	uuid (FK -> project)	所属项目，ON DELETE CASCADE
name	varchar(255)	文档名称
type	varchar(32)	类型：`prd`，`figma`，`api_spec`
url	text	对象存储路径或外部链接
content_text	text	提取的原始文本（可选，用于调试或全文检索）
metadata	jsonb	文档元数据（Figma版本、PRD作者等）
created_at	timestamptz	
updated_at	timestamptz	

索引：`idx_document_project_id`，`idx_document_type`。

`document_chunk` 文档分块表

字段	类型	说明
id	uuid (PK)	主键，同时作为Milvus中对应向量的ID
document_id	uuid (FK -> document)	所属文档，ON DELETE CASCADE
chunk_index	integer	块序号
content	text	分块文本
metadata	jsonb	元数据（如章节标题、Figma节点路径）
created_at	timestamptz	（无updated_at，因为不可修改）

索引：`idx_document_chunk_document_id`。

`generation_task` 生成任务表

字段	类型	说明
id	uuid (PK)	
project_id	uuid (FK -> project)	所属项目，ON DELETE CASCADE
user_id	uuid (FK -> users)	发起人，ON DELETE NO ACTION
status	varchar(32)	`pending`，`processing`，`completed`，`failed`
prompt	text	实际发送给大模型的提示词（含检索上下文）
result_summary	jsonb	结果摘要（生成用例数、引用文档数等）
error_msg	text	失败原因

created_at	timestamptz	
updated_at	timestamptz	

索引： `idx_generation_task_project_id`，`idx_generation_task_user_id`，`idx_generation_task_status`。

`generated_case_draft` 生成用例草稿表

字段	类型	说明
id	uuid (PK)	
task_id	uuid (FK -> generation_task)	所属生成任务，ON DELETE CASCADE
module_id	uuid (FK -> module)	用户确认时关联的模块（可为空），ON DELETE SET NULL

title	varchar(255)	标题
preconditions	jsonb	前置条件数组
steps	jsonb	步骤数组
expected	jsonb	预期结果对象
case_type	case_type_enum	用例类型
priority	priority_enum	优先级
ai_metadata	jsonb	记录引用的`document_chunk.id`列表、模型信息等
status	varchar(32)	`pending`, `confirmed`, `rejected`
feedback	text	用户拒绝时的反馈
created_at	timestamptz	
updated_at	timestamptz	

索引：`idx_draft_task_id`, `idx_draft_module_id`, `idx_draft_status`。

---

4. 核心模块设计

4.1 用户与权限模块（user-service）

- 基于JWT的认证，支持角色权限控制。
- 提供用户注册、登录、信息修改。
- 项目成员管理：每个项目可独立添加成员，赋予项目内角色（管理员/工程师/访客），与全局角色叠加。

4.2 项目管理模块（project-service）

功能：

- 项目CRUD，支持归档。
- 模块树形管理，模块名称在项目内唯一。
- **项目配置管理**：
  - 支持键值对配置的增删改查（`project_config`表）。
  - 典型配置项：大模型参数、环境变量、通知设置等。
  - 配置值使用JSONB，支持复杂结构。

- 项目级配置可通过缓存（如Redis）加速读取。

4.3 文档知识库模块（document-service）

功能：

- 上传文档（PRD/PDF/Word/Figma URL）到对象存储，创建`document`记录。
- 异步解析文档：
  - PRD：调用文本抽取库（如`unstructured`）提取纯文本，按标题分块。
  - Figma：通过Figma API获取文件JSON，遍历节点树，提取文本节点及路径，生成块。
  - API Spec：解析OpenAPI JSON，提取接口描述、参数、示例。
- 为每个分块生成embedding（调用LLM embedding API），将向量存入Milvus，分块元数据存入`document_chunk`。
- 支持文档版本更新：重新解析并替换原有分块（先删除旧块再插入新块）。

关键接口：

- `POST /api/v1/projects/{id}/documents`：上传文档（返回任务ID）。
- `GET /api/v1/documents/{id}`：获取文档状态及分块概览。

4.4 RAG检索模块（rag-service）

功能：

- 接收查询文本，调用embedding模型转为向量。
- 在Milvus中执行向量相似度检索（余弦距离），返回Top-K的`document_chunk.id`。
- 根据ID从PostgreSQL获取完整chunk内容及文档元数据。
- 支持按文档类型、项目ID过滤。

接口（gRPC）：

protobuf

```
service RAGService {
  rpc Retrieve(RetrieveRequest) returns (RetrieveResponse);
}
message RetrieveRequest {
  string query = 1;
  string project_id = 2;
  int32 top_k = 3;
  repeated string doc_types = 4;
}

message Chunk {
  string id = 1;
```

```
  string id = 1;
  string document_id = 2;
  string content = 3;
  map<string, string> metadata = 4;
  float score = 5;
}
```

4.5 用例生成模块（generation-service）

功能：

- 接收用户发起生成任务（指定项目、模块、需求描述等），创建`generation_task`记录。
- 调用RAG服务检索相关文档片段，组装提示词。
- 异步调用大模型（通过消息队列）生成用例，解析返回的JSON，存入`generated_case_draft`。
- 提供草稿确认/拒绝接口：确认时，将草稿数据转入`test_case`表，生成唯一编号，并关联引用块到`ai_metadata`。

提示词设计示例：

text

```
你是一个资深的测试工程师。请根据以下产品需求片段，生成5个测试用例，覆盖正向、异常、边界场景。
需求片段：
{chunk1}
{chunk2}
...
请以JSON数组格式返回，每个用例包含：title, preconditions(array), steps(array),
expected(object), type(enum), priority(enum)。
```

4.6 测试用例管理模块（testcase-service）

- 标准CRUD，支持批量导入导出（Excel/CSV）。
- 用例版本历史：每次更新保存快照（可基于`updated_at`触发或显式创建版本记录）。
- 需求追溯：通过`ai_metadata.referenced_chunks`关联到知识库块，界面可展示来源。
- 草稿管理：提供草稿列表、确认、拒绝接口。

4.7 测试计划与执行模块（testplan-service）

- 计划创建时，支持从用例库中选择用例，或根据需求变更智能推荐。
- 分配执行人，状态流转。

- 执行结果录入：支持Web表单填写，也可通过API接入自动化测试框架（如Jenkins回调）。
- 执行历史查询，每个用例的最新状态同步更新（可选）。

---

5. 接口设计（RESTful API概览）

所有API以`/api/v1`为前缀，返回JSON格式，认证通过JWT Header `Authorization: Bearer <token>`。

模块	端点	方法	说明
项目	`/projects`	GET/POST	项目列表/创建
	`/projects/{id}`	GET/PUT/DELETE	项目详情/更新/删除
模块	`/projects/{id}/modules`	GET/POST	模块列表/创建
	`/modules/{id}`	GET/PUT/DELETE	模块详情/更新/删除
项目配置	`/projects/{id}/configs`	GET/POST	配置列表/创建
	`/projects/{id}/configs/{key}`	GET/PUT/DELETE	配置项操作
用户	`/users`	GET/POST	用户列表/创建（仅管理员）
	`/users/{id}`	GET/PUT/DELETE	用户操作
文档	`/projects/{id}/documents`	POST	上传文档（表单）
	`/documents/{id}`	GET	获取文档状态
	`/documents/{id}/chunks`	GET	获取分块列表（调试）
生成任务	`/projects/{id}/generation/tasks`	POST	发起生成任务
	`/generation/tasks/{id}`	GET	查询任务状态及生成的草稿列表
草稿	`/generation/drafts/{id}/confirm`	PUT	确认草稿（需传入module_id）
	`/generation/drafts/{id}/reject`	PUT	拒绝草稿（可传反馈）
测试用例	`/cases`	GET	用例列表（支持分页、过滤）
	`/cases`	POST	创建用例（手动）
	`/cases/{id}`	GET/PUT/DELETE	用例详情/更新/删除
测试计划	`/plans`	GET/POST	计划列表/创建
	`/plans/{id}`	GET/PUT/DELETE	计划详情/更新/删除
	`/plans/{id}/cases`	POST	关联用例
测试结果	`/results`	POST	录入执行结果
	`/cases/{id}/results`	GET	获取用例的所有执行历史

---

6. 核心业务流程

6.1 文档上传与知识库构建

6.1 文档上传与知识库构建

1. 用户上传文件或提供Figma URL。
2. 文档服务接收文件，存入OSS，创建`document`记录（状态：pending）。
3. 将解析任务推入消息队列。
4. 消费者从队列拉取任务：
    - 根据文档类型调用对应解析器，生成分块列表。
    - 为每个分块调用embedding接口，获取向量。
    - 向量存入Milvus，分块信息存入`document_chunk`。
    - 更新`document`状态为completed。
5. 完成后可发送通知。

6.2 AI用例生成与确认

1. 用户在项目下发起生成任务（指定模块、需求描述等）。
2. 生成服务创建`generation_task`（status=pending），推入队列。
3. 消费者处理：
    - 调用RAG服务检索相关文档块。
    - 构建提示词，调用大模型。
    - 解析返回的JSON，逐条插入`generated_case_draft`（status=pending）。
    - 更新任务状态为completed，记录result_summary。
4. 用户进入草稿箱，查看生成的用例，可编辑、确认或拒绝。
    - 确认时，将草稿数据复制到`test_case`，生成编号（如`PRJ-{module}-{yyyymmdd}-{seq}`），`ai_metadata`记录引用的chunk ID。
    - 拒绝时，记录反馈，草稿状态置为rejected。
5. 确认后的用例进入正式库，可参与后续测试计划。

6.3 测试计划与执行

1. 创建计划，从用例库中选择用例（可基于AI推荐）。
2. 分配执行人，计划状态变为active。
3. 执行人在Web界面录入执行结果（pass/fail/block/skip），可附日志截图。
4. 系统记录`test_result`，可选更新`test_case.status`。
5. 计划内所有用例执行完毕，计划状态自动变为completed。

---

7. 部署架构

7.1 容器化部署

- 所有Go服务打包为Docker镜像，通过Kubernetes部署。
- PostgreSQL可选用云数据库RDS或自建主从（注意索引优化）。
- Milvus集群部署（至少1个Coordinator + 多个QueryNode/DataNode）。
- MinIO作为对象存储，支持多副本。
- 消息队列NSQ集群（或RabbitMQ）。
- 监控：Prometheus + Grafana，日志：Fluentd + Elasticsearch + Kibana。

7.2 扩展性设计

- **无状态服务**（API网关、各业务服务）可水平扩展。
- **有状态服务**：PostgreSQL采用读写分离；Milvus支持水平分片；MinIO本身支持分布式。
- **异步任务**：通过消息队列削峰填谷，任务消费者可独立扩缩容。
- **大模型调用**：设置熔断、重试机制，避免因模型超时而阻塞。

7.3 环境隔离

- 开发、测试、生产环境独立K8s命名空间。
- 每个租户（项目）数据逻辑隔离（通过`project_id`过滤），可考虑未来多租户物理隔离。

---

8. 关键技术点与最佳实践

8.1 Go编码规范

- 项目采用标准布局（`/cmd`, `/internal`, `/pkg`, `/api`）。
- 使用Wire进行依赖注入，提升可测试性。
- 错误处理：统一错误码，使用`github.com/pkg/errors`包装堆栈。
- 日志：使用zap结构化日志，输出JSON格式。
- 配置管理：Viper读取环境变量或配置文件，支持从`project_config`动态加载。

8.2 数据库操作

- 使用GORM作为ORM，自动迁移表结构（生产环境需手动控制）。
- 对于JSONB字段，定义Go结构体，利用GORM的`serializer:json`标签。
- 复杂查询使用原生SQL（sqlx）提升性能。
- **项目配置缓存**：高频读取的配置可缓存到Redis，减少数据库压力。

8.3 异步任务处理

8.3 异步任务处理

- 使用NSQ作为轻量级消息队列，消费者并发处理。
- 任务定义：每个任务包含类型和载荷（JSON），如`DocumentParseTask`, `GenerationTask`。
- 失败重试：设置最大重试次数，超过则写入死信队列并告警。

8.4 安全性

- 密码使用bcrypt加密存储。
- JWT密钥定期轮换，设置短过期时间（如2h）。
- API敏感操作记录审计日志（操作人、时间、资源、结果）。
- 对象存储URL生成带时效的签名，防止未授权访问。
- 项目配置中的敏感值（如API Key）应加密存储（应用层加密）。

8.5 可观测性

- 指标：每个服务暴露Prometheus端点，记录QPS、延迟、错误数、数据库连接池状态。
- 链路追踪：集成OpenTelemetry，通过Jaeger展示调用链。
- 日志：统一格式，包含`trace_id`便于关联。

8.6 针对`project_config`的优化

- 读取配置时，可批量获取项目所有配置，减少多次查询。
- 写操作时，注意更新缓存。
- 配置变更可发布事件，供其他服务感知。

---

9. 未来演进方向

- **多模态测试生成**：利用视觉模型分析Figma截图，自动生成UI自动化脚本（如Playwright）。
- **测试资产血缘图谱**：构建需求-用例-缺陷的完整链路，支持变更影响分析。
- **智能缺陷定位**：结合代码库，将失败用例自动关联到可疑代码提交。
- **联邦学习**：在企业私有化场景下，利用联邦学习提升模型效果而不共享数据。